Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

Judul Paper IEEE Access!

YASIN ARIL MUSTOFA[1,3], (Member, IEEE), SYAFRUDDIN SYARIF[1], AND MUHAMMAD NISWAR[2]
[1]Department of Electrical Engineering, Hasanuddin University, Makassar, Indonesia
[2]Department of Informatics, Hasanuddin University, Makassar, Indonesia
[3]Department of Informatics, Faculty of Computer Science, Ichsan Sidenreng Rappang University, Sidenreng Rappang, Indonesia

Corresponding author: First A. Author (e-mail: author@ boulder.nist.gov).

This paragraph of the first footnote will contain support information, including sponsor and financial support acknowledgment. For example, "This work was supported in part by the U.S. Department of Commerce under Grant BS123456."

ABSTRACT These instructions give you guidelines for preparing papers for IEEE Access. Use this document as a template if you are using LaTeX. Otherwise, use this document as an instruction set. The electronic file of your paper will be formatted further at IEEE. Paper titles should be written in uppercase and lowercase letters, not all uppercase. Avoid writing long formulas with subscripts in the title; short formulas that identify the elements are fine (e.g., "Nd–Fe–B"). Do not write ''(Invited)'' in the title. Full names of authors are preferred in the author field, but are not required. Put a space between authors' initials. The abstract must be a concise yet comprehensive reflection of what is in your article. In particular, the abstract must be self-contained, without abbreviations, footnotes, or references. It should be a microcosm of the full article. The abstract must be between 150–250 words. Be sure that you adhere to these limits; otherwise, you will need to edit your abstract accordingly. The abstract must be written as one paragraph, and should not contain displayed mathematical equations or tabular material. The abstract should include three or four different keywords or phrases, as this will help readers to find it. It is important to avoid over-repetition of such phrases as this can result in a page being rejected by search engines. Ensure that your abstract reads well and is grammatically correct.

INDEX TERMS Enter key words or phrases in alphabetical order, separated by commas. Autocorrelation, beamforming, communications technology, dictionary learning, feedback, fMRI, mmWave, multipath, system design, multipath, slight fault, underlubrication fault.

I. INTRODUCTION

Edge networks, content delivery networks (CDNs), and information centric networking (ICN) architectures increasingly rely on caching to reduce end-to-end latency and origin load, making cache effectiveness a core systems concern rather than a purely local optimization. Cache performance is commonly summarized through hit probability or hit ratio, namely the fraction of requests served from cache instead of the origin, and this metric directly shapes end-to-end delivery efficiency (Panigrahy et al., 2021). In practice, cache spaces remain small relative to modern content catalogs, so cache outcomes depend critically on what is admitted into the cache and what is later evicted as requests arrive (Aboodi et al., 2019; Hou et al., 2023; Wang & Friderikos, 2020; Batool et al., 2024). LRU eviction remains a widely used baseline in networked caching because it is simple and effective under favorable locality assumptions, yet its behavior can become brittle when these assumptions are violated, including regimes with asymmetric object sizes that degrade LRU's space utilization (Friedlander & Aggarwal, 2019). As caching is deployed closer to users and workloads diversify, the need to understand, control, and improve hit ratio under realistic demand dynamics becomes increasingly urgent (Wang & Friderikos, 2020; Batool et al., 2024).

A central practical challenge is that content demand is frequently nonstationary: popularities change over time, transient items appear and disappear, and user preferences evolve, all of which can reduce hit ratio and increase wasted cache occupancy unless policies adapt (Wu et al., 2019; Qi et al., 2019; Sun et al., 2019; Banerjee & Biswash, 2024). Under dynamic popularity, a cache can end up serving "yesterday's" hot set rather than "today's," producing a mismatch between the cached set and near-future requests that reduces hit ratio and motivates predictive or learning-based updates (Wu et al., 2019; Qi et al., 2019; Sun et al., 2019; Wang & Friderikos, 2020). This mismatch also induces cache pollution, understood as cache space consumed by items that provide little benefit and displace higher-value objects; pollution is explic-

itly defined as wasting space on objects that are no longer accessed, thereby degrading cache effectiveness (Kalambe et al., 2019). The risk is amplified in environments with transient content where relevance is short-lived, such as vehicular and IoT-like settings, in which admitting content unlikely to be reused before it becomes irrelevant directly translates into wasted occupancy and degraded performance (Amadeo, 2021; Dhawan et al., 2023). Moreover, in-network caches face additional "pollution-like" waste through redundancy, where storing duplicate replicas across nodes reduces effective diversity and can reduce overall hit probability, motivating redundancy-avoidance strategies (Nguye n et al., 2019; Chang et al., 2022; Batool et al., 2024). When combined with aggressive admit-on-miss behavior under LRU eviction, these conditions can create churn and stale occupancy, and can even trigger size-driven pathologies such as looped replacement with large objects, which wastes cache usage and degrades hit probability (Yoji et al., 2020; Friedlander & Aggarwal, 2019). Consequently, improving hit ratio in realistic systems is not only a matter of choosing a stronger eviction heuristic, but also of ensuring that the admission process does not inject low-benefit items that fuel pollution, churn, and redundancy (Batool et al., 2024; Rashid et al., 2021).

Recent research reflects this landscape through several families of approaches. One family emphasizes learning-driven selection of what to cache by estimating popularity or demand and using that estimate to guide caching decisions under dynamic conditions, aiming to outperform purely reactive heuristics (Hou et al., 2023; Wang et al., 2020). A second family focuses on learned or benefit-driven admission mechanisms that decide whether to insert an object at all, motivated by operational constraints where write traffic, backend load, or device wear makes indiscriminate insertion undesirable; prominent examples include benefit-optimized admission in production flash caches (Yang et al., 2023) and learned reuse-time admission in CDN caches that reports hit-rate gains together with reduced write traffic (Liu et al., 2024). A third family combines selectivity with dynamic demand handling through hybrid or gated designs, such as probabilistic insertion in hybrid proactive–reactive edge caching validated on real datasets (Qi et al., 2019), demand-forecasting-based selective caching on real content data (Hoiles et al., 2021), threshold-based admission that adjusts as popularity changes (Banerjee & Biswash, 2024), and admission rules that explicitly mitigate churn pathologies like looped replacement to improve hit probability (Yoji et al., 2020). Across these directions, the literature consistently highlights the difficulty of sustaining cache effectiveness under time-varying demand and the need for mechanisms that avoid wasting scarce cache capacity while improving hit-oriented outcomes (Wang & Friderikos, 2020; Batool et al., 2024).

Despite this progress, a gap remains in how score-based or learning-assisted admission is stabilized and interpreted under realistic nonstationary workloads. Many works motivate improvements through "better selection quality," where learning is presumed to track changing popularity and therefore cache higher-utility objects (Hou et al., 2023; Wu et al., 2019; Sun et al., 2019; Wang et al., 2020). However, the empirical separation between gains from better selection and gains from reduced insertion volume is often incomplete, because many evaluations do not report insertion intensity, churn, or direct pollution indicators even though pollution and wasted occupancy are central concerns (Kalambe et al., 2019; Rashid et al., 2021). Moreover, learned admission presumes that scores or predicted reuse signals are sufficiently reliable at the time decisions are made; under nonstationarity, learning lag and finite-time effects can cause estimates to become stale, a phenomenon studied as regret in online learning analyses (Bura et al., 2022) and as performance limits for non-anticipative reactive caching policies (Panigrahy et al., 2021). In streaming machine learning, the broader notion that distributional changes can degrade predictive accuracy is commonly discussed as concept drift (Li et al., 2022; Hashmani et al., 2019), which aligns with how caching studies describe time-varying popularity and transient relevance even when they do not use drift terminology (Wu et al., 2019; Qi et al., 2019; Amadeo, 2021; Dhawan et al., 2023). In practice, this means that score-based admission can become unstable or overly permissive when ranking information is ambiguous, producing insertion bursts that increase pollution and churn under LRU eviction, and thereby erode hit ratio even if an incremental predictor exists (Kalambe et al., 2019; Yoji et al., 2020; Rashid et al., 2021). What is missing is a clearly defined stabilization layer that can be audited as an admission-only mechanism, holds eviction fixed for attribution, and validates its causal role using explicit insertion and pollution measures, not only hit ratio.

This paper addresses that gap by proposing a stabilization layer for proactive, score-based admission that explicitly regulates admission pressure using signals tied to decision reliability, while keeping eviction fixed to LRU to ensure interpretability. Our setting follows the modular view that caching performance depends jointly on placement or admission and on eviction or replacement, and that these modules can be evaluated separately (Batool et al., 2024; Rashid et al., 2021). We assume an upstream scoring backbone, such as an incremental learner that produces per-candidate scores over time (Elwell and Polikar, 2011; Xu et al., 2020), but we focus on the downstream question that determines cache safety in deployment: how to prevent unreliable or low-separation scores from triggering overly aggressive insertions that pollute the cache. The key idea is to convert "take the top scored candidates" into a controlled admission process that uses feedback from realized outcomes and a quality indicator of score separability to gate insertion. This approach is aligned with evidence that insertion control can improve hit probability by mitigating churn and waste, whether through probabilistic throttling on real traces (Qi et al., 2019), admission rules that prevent wasteful replacement loops (Yoji et al., 2020), reuse-detection style insertion filtering (Díaz et al., 2019), or operational admission systems that explicitly optimize cost and write traffic alongside hit outcomes (Yang

et al., 2023; Liu et al., 2024). It is also practically motivated by the need to maintain high forwarding throughput and avoid unnecessary processing in networked caches, where overly complex per-request logic can be undesirable (Takemasa et al., 2019). In this paper, we refer to the proposed set of stabilization safeguards for score-based admission, including effective-precision feedback and score-quality gating, as guardrails.

Our main contributions are as follows:

1) We introduce Guardrails Admission, a stabilization mechanism for proactive cache admission under fixed LRU eviction that regulates admission pressure using (i) outcome-based effective-precision feedback and (ii) score-quality gating, improving hit ratio while reducing cache pollution and unnecessary insertions.
2) We conduct a component-wise ablation and mechanism analysis demonstrating that the guardrails elements are complementary: removing either effective-precision feedback or score-quality gating degrades hit ratio and/or increases pollution indicators, validating the necessity of the full stabilization design under fixed eviction.

The remainder of the paper is organized as follows. Section II reviews related work on caching under dynamic demand, learning-assisted admission, and pollution mitigation. Section III introduces the system model and metrics, including insertion and pollution indicators and admission efficiency. Section IV formulates the admission-stabilization problem under fixed LRU eviction. Section V presents Guardrails Admission and its two complementary safeguards. Section VI reports trace-driven evaluations against representative baselines and includes ablation and mechanism analyses. Section VII concludes with a summary and directions for future work.

II. RELATED WORK

Proactive caching has been studied across diverse deployment contexts, including mobile edge computing, CDNs, and ICN-style in-network caching, where the goal is to anticipate demand and place content closer to users to improve service metrics. A common instantiation is prefetching or pre-positioning, which leverages forecasts of near-future requests to fetch content before explicit demand arrives, often motivated by latency and quality-of-experience objectives in edge-assisted delivery (Aguilar-Armijo et al., 2023; Wang & Friderikos, 2020). Within this predictive family, learned demand or popularity forecasting from historical requests and content features is widely used to produce ranked candidate sets for placement decisions, including risk-aware designs that incorporate uncertainty in forecast distributions (Sun et al., 2019; Hoiles et al., 2021; Wang et al., 2020). At the architectural level, these proactive methods operate over multiple decision levers, spanning prefetching, admission, and replacement, and they are commonly analyzed through the modular view that cache performance depends jointly on what enters the cache and what is later evicted (Batool et al., 2024; Rashid et al., 2021). Consequently, the literature encompasses both end-to-end network-wide placement frameworks and single-cache mechanisms that treat prediction as an upstream signal rather than a complete caching policy.

A recurring theme across edge and ICN caching studies is that the request process is nonstationary: content popularities vary over time, transient contents appear, and user preferences shift, creating a persistent mismatch between cached contents and current demand if the policy does not adapt (Wu et al., 2019; Qi et al., 2019; Banerjee & Biswash, 2024). This mismatch has two intertwined consequences. First, hit performance degrades during transitions because reactive policies effectively track past popularity, a limitation that is reinforced by theoretical analyses of non-anticipative caching and by online-learning viewpoints that stress finite-time learning speed (Panigrahy et al., 2021; Bura et al., 2022). Second, nonstationarity increases cache pollution risk, where space is consumed by objects that deliver little benefit; pollution is explicitly framed as wasted occupancy by objects no longer accessed, which is particularly salient under transient content relevance in vehicular and IoT-like workloads (Kalambe et al., 2019; Amadeo, 2021; Dhawan et al., 2023). In networked caches, redundancy further acts as a pollution-like channel by reducing effective diversity when duplicates occupy multiple caches along a path (Nguyen et al., 2019). These challenges are amplified by heterogeneity and large-object effects that can induce replacement churn and wasteful cache usage under common eviction mechanisms (Friedlander & Aggarwal, 2019; Yoji et al., 2020).

Reinforcement Learning, Online Learning, and Drift Handling. To cope with time-varying demand, adaptive caching mechanisms can be grouped into three broad families. First, reinforcement learning methods treat cache update as a sequential decision problem and learn a policy that maps observed request history and cache state to caching actions, reporting improved hit rate relative to classical heuristics in dynamic popularity settings (Wu et al., 2019; Hou et al., 2023). Second, online and incremental learning methods continuously update predictors or statistics from streaming arrivals and use the resulting scores to drive placement or admission decisions. Examples range from incremental popularity estimators that use bounded histories to respond promptly to changes (Wang et al., 2020), to lightweight frequency-window admission filters designed for low overhead in routers (Takemasa et al., 2019), to learned reuse-time admission that conditions decisions on operating conditions such as cache size and reports hit-rate gains on production traces (Liu et al., 2024). Industrial systems similarly optimize admission based on observed benefit signals and operational objectives in production flash caches (Yang et al., 2023). Third, drift detection and concept drift handling are emphasized in streaming machine learning as a general response to distributional change, where model accuracy can degrade unless drift is monitored and adaptation safeguards are employed (Li et al., 2022; Hashmani et al., 2019). While caching papers often motivate adaptation through dynamic popularity rather than formal drift detection, incremental learners used in

streaming contexts, such as Learn++.NSE, provide a canonical backbone for continual updates that can be incorporated upstream of caching decisions (Elwell and Polikar, 2011; Xu et al., 2020).

Closest Prior to Guardrails. The works most closely aligned with this paper are those that treat admission as an explicit lever to control insertion intensity and reduce waste, especially when eviction is kept conventional. Learned admission policies exemplify this direction by predicting reuse characteristics and admitting only objects expected to be reused within an effective horizon, with evaluations that jointly report hit rate and insertion-related costs such as write traffic (Liu et al., 2024). Benefit-driven admission optimization in production further supports admission as a primary control knob when operational costs and device wear are central (Yang et al., 2023). In edge and ICN settings, admission throttling appears as probabilistic caching in hybrid proactive-reactive designs under dynamic popularity, where selective insertion improves cache-served traffic metrics relative to reactive baselines on real datasets (Qi et al., 2019), and as adaptive popularity thresholds that gate caching actions over time (Banerjee & Biswash, 2024). Complementary evidence comes from mechanisms that explicitly target insertion-driven churn pathologies, such as frequent-arrival admission to mitigate looped replacement in packet-level caching (Yoji et al., 2020), and from reuse-detection style insertion filtering as a clean admission-only template (Díaz et al., 2019). In parallel, pollution-mitigation research often focuses on replacement criteria that integrate size, frequency, and recency to reduce wasted occupancy (Kalambe et al., 2019) or on maturity-aware eviction policies designed for changing popularity regimes (Rashid et al., 2021), but these approaches generally alter eviction logic rather than stabilizing score-based admission under a fixed eviction baseline. Across the admission-centric works, the gating mechanism is typically driven by a single predictor or threshold, whereas the reliability of the score signal itself and the stability of insertion behavior are less often treated as first-class control objectives.

Taken together, prior work establishes that learning can improve cache outcomes by improving selection quality and that admission throttling can reduce waste and churn, yet the literature still leaves an attribution and stabilization gap for score-based admission in nonstationary regimes. Specifically, many studies report hit(-related) improvements without explicitly separating whether gains arise from better ranking versus from reduced insertion pressure, even though pollution is formally defined as wasted occupancy and churn mechanisms are known to degrade hit probability under aggressive insertion (Kalambe et al., 2019; Yoji et al., 2020). Moreover, while online learning and incremental predictors are natural upstream components under changing demand (Bura et al., 2022; Xu et al., 2020), and drift handling is well studied in streaming ML (Li et al., 2022; Hashmani et al., 2019), existing caching evaluations rarely provide an auditable stabilization layer that regulates admission based on decision outcome feedback and score-signal quality, holds eviction fixed for clean attribution, and validates component necessity through controlled ablations. Accordingly, the remaining gap addressed in this paper is a dedicated admission-only stabilization mechanism, termed guardrails, that converts score-based proactive admission into a feedback- and quality-gated process under fixed LRU eviction and quantifies its causal effect through insertion stability, pollution reduction, and component-wise ablation on real request traces.

III. PRELIMINARIES

This section formalizes the online caching setting in which Guardrails Admission operates and fixes the notation used in the remainder of the paper. We adopt a trace-driven view where a request stream is replayed against a cache model and each request produces a binary hit or miss outcome, since cache hit ratio (or hit probability) is the canonical summary metric of cache effectiveness in such evaluations (Panigrahy et al., 2021).

A. ONLINE REQUEST STREAM, SLOTTING, AND ADMISSION CANDIDATES

We model a request trace as a time-ordered sequence of request events $\{(o_i, t_i)\}_{i=1}^{N}$, where o_i denotes the requested object identifier and t_i is the request timestamp. For online control and batched scoring, time is partitioned into consecutive slots indexed by $t \in \{0, 1, 2, \ldots\}$, where slot t corresponds to a half-open interval $[T_t, T_{t+1})$. The requests observed in slot t are collected as $\mathcal{R}_t = \{(o_i, t_i) \mid t_i \in [T_t, T_{t+1})\}$. A slot may represent a fixed wall-clock duration or a fixed number of requests; the notation is agnostic to this choice, which is specified in the experimental setup. Although $\mathcal{R}_t$ can be treated as an ordered list, we use set notation when only membership matters. We use "end of slot t" to denote the instant immediately after processing the last request with $t_i \in [T_t, T_{t+1})$. Slotting is a modeling convenience: requests are served continuously within the slot, while statistics for learning and admission are aggregated and consumed at the slot boundary. This timing discipline clarifies what information is available when an admission decision is made.

Within each slot, objects requested on cache misses form the admission candidate pool. Let $\mathcal{C}_t$ denote the set of unique objects that experience at least one miss in slot t: $\mathcal{C}_t = \{o_i \mid \exists (o_i, t_i) \in \mathcal{R}_t$ such that o_i is a miss at $t_i\}$. If an object misses multiple times inside the same slot, it still appears once in $\mathcal{C}_t$; we maintain auxiliary aggregates such as the per-slot miss count $m_t(o)$ and recency features for learning and ranking. This definition yields a candidate set at object granularity, and $\mathcal{C}_t$ is the only pool from which proactive admission may draw. All features, scores, and control signals used to decide A_t are required to be online-safe: computations at slot t can depend only on the request history up to the end of slot t and on the current cache state, with no look-ahead into future requests. This causality constraint matches the non-anticipative setting assumed in trace-driven caching analyses (Panigrahy et al., 2021).

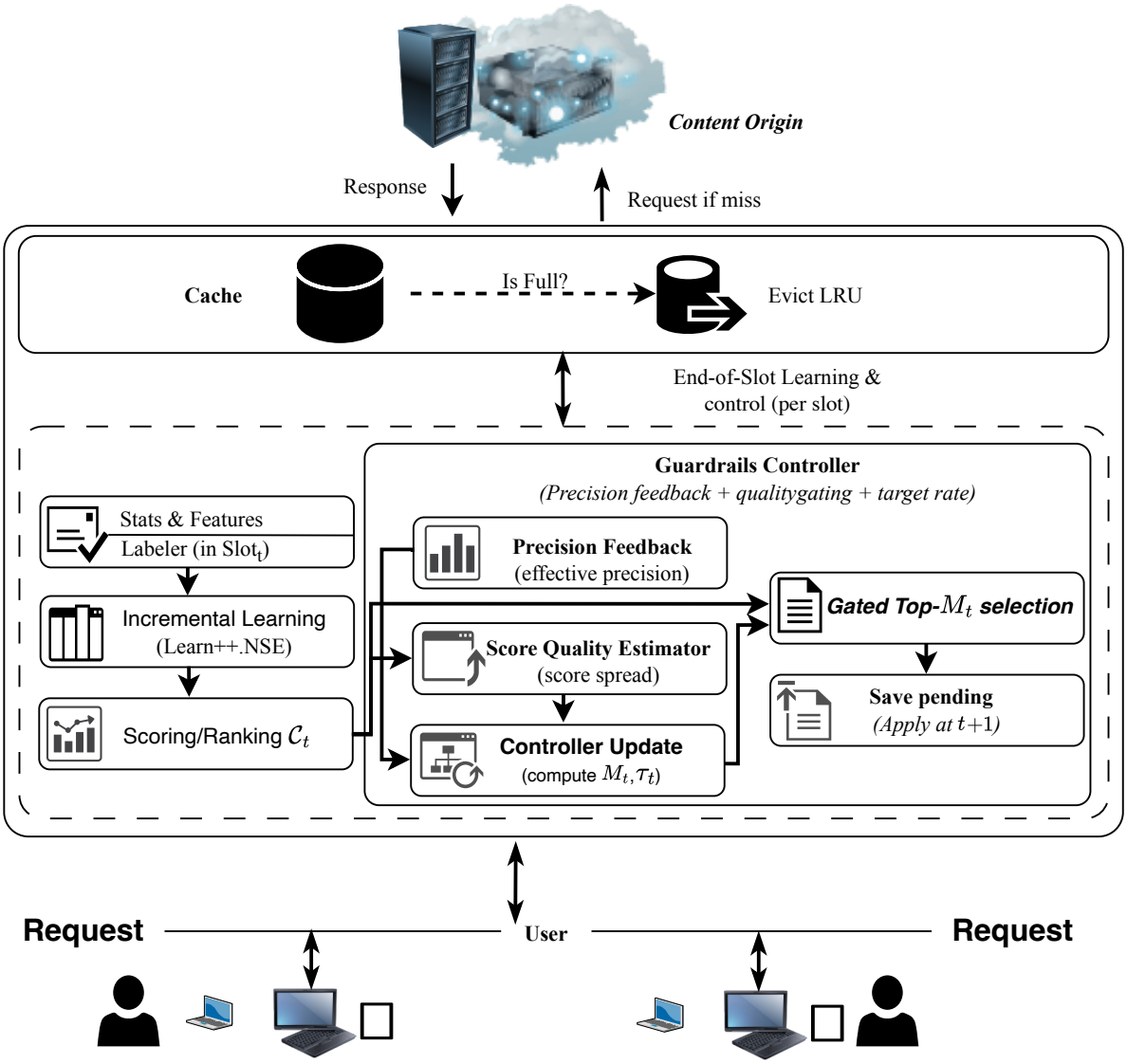

FIGURE 1. System architecture and cache-control modules at an edge node

B. CACHE DYNAMICS UNDER FIXED LRU AND ONE-SLOT DELAYED ADMISSION

We consider an object cache with capacity C (measured in number of objects) and LRU eviction. Let S_t denote the cache state at the beginning of slot t, represented as an ordered list of resident objects from most recently used (MRU) to least recently used (LRU). For a request $(o_i, t_i) \in \mathcal{R}_t$, the cache returns a hit if $o_i \in S_t$ at the request time; on a hit, LRU is updated by moving o_i to the MRU position. On a miss, the object is fetched from the origin and becomes eligible for admission, but it is not inserted unless selected by the admission controller. Our evaluation and definitions are at object granularity, so a hit means the entire object is served from cache; this avoids partial-hit nuances that arise in chunk-based or packet-level caching (Friedlander & Aggarwal, 2019; Yoji et al., 2020). Keeping eviction fixed to LRU ensures that any performance differences can be attributed to admission decisions rather than replacement changes.

Admission is decided at slot boundaries. At the end of slot t, an admission controller selects a subset $A_t \subseteq \mathcal{C}_t$ of candidates that should be inserted into the cache. We adopt a one-slot delayed apply rule to separate online request processing from end-of-slot computation: we compute A_t from $\mathcal{C}_t$ at the end of slot t and apply these insertions at the beginning of slot $t+1$. Consequently, the insertions that occur during slot t are precisely A_{t-1}^{applied}, which were computed from the miss candidates $\mathcal{C}_{t-1}$. Let A_t^{applied} denote the set that is actually inserted at the start of slot $t+1$; in our design $A_t^{\text{applied}} = A_t$ because no further filtering occurs between decision and application. When inserting A_t^{applied}, the cache follows standard LRU mechanics: each inserted object is placed at MRU, and if capacity is exceeded, evictions remove LRU objects until

$|S_{t+1}| \leq C$. This execution order is the basis for defining insertion intensity and outcome feedback, while keeping eviction logic unchanged across all experiments.

C. METRICS FOR HIT RATIO, INSERTION STABILITY, AND POLLUTION

The primary effectiveness metric is cache hit ratio. For each request event (o_i, t_i), let $H_i \in \{0,1\}$ be the hit indicator, where $H_i = 1$ if the request is served from cache and $H_i = 0$ otherwise. Over a trace of N requests, the request-level hit ratio is $\mathrm{HR} = \frac{1}{N}\sum_{i=1}^{N} H_i$, which matches the standard definition of hit probability as the fraction of requests served from cache in trace-driven evaluation (Panigrahy et al., 2021). The complementary miss ratio is $1 - \mathrm{HR}$, but our analysis emphasizes hit ratio because it directly reflects cache-served demand and is the main objective in dynamic caching policy evaluations (Wu et al., 2019). Because our admission controller operates at slot boundaries, we also use a per-slot view: $\mathrm{HR}_t = \frac{1}{|\mathcal{R}_t|}\sum_{(o_i,t_i)\in\mathcal{R}_t} H_i$. This per-slot accounting supports temporal analyses by revealing when hit ratio degrades or improves as admission decisions adapt over time. Unless noted otherwise, all hit metrics are computed at the same object granularity used by the cache model, so each hit corresponds to an entire object being served from cache.

To quantify insertion stability, we measure how aggressively the cache state is modified by admissions. The insertion count applied at the beginning of slot t is $I_t = |A_{t-1}^{\mathrm{applied}}|$, and a normalized admission rate is $\rho_t = \frac{|A_t|}{\max\{1,|\mathcal{C}_t|\}}$, namely the fraction of miss candidates admitted at the end of slot t. These quantities capture admission pressure and help distinguish selection-quality effects from admission-volume effects when comparing policies. Admission-centric studies often report insertion volume via write traffic or device-oriented proxies, because insertions translate into cache writes and churn. For example, learned CDN admission reports paired improvements in hit rate and reductions in write traffic on production traces (Liu et al., 2024), and datacenter flash cache admission evaluates reductions in disk reads and flash wearout under production experiments (Yang et al., 2023). In our trace-driven setting, I_t and ρ_t serve as the primary insertion-stability measures and can be converted to byte-level traffic when object sizes are known.

We further measure admission efficiency and pollution using outcome-based proxies computed from the same replay. Let $\mathrm{Hits}(A_{t-1}^{\mathrm{applied}}; W)$ denote the number of cache hits generated by objects admitted from $A_{t-1}^{\mathrm{applied}}$ within a post-admission horizon of W slots; W is specified in Section VI. The hit-yield per insertion is $Y_t = \frac{\mathrm{Hits}(A_{t-1}^{\mathrm{applied}};W)}{\max\{1,|A_{t-1}^{\mathrm{applied}}|\}}$, which captures how many hits are obtained per admitted object. To quantify pollution, we mark an admitted object as non-beneficial if it produces zero hits within the same horizon. Define $Z_t(o) = 1$ if $o \in A_{t-1}^{\mathrm{applied}}$ and o yields no hits within W slots after insertion, and $Z_t(o) = 0$ otherwise. Our pollution proxy is the never-reused admission rate $\mathrm{Poll}_t = \frac{\sum_{o\in A_{t-1}^{\mathrm{applied}}} Z_t(o)}{\max\{1,|A_{t-1}^{\mathrm{applied}}|\}}$, which operationalizes pollution as admissions that consume cache capacity yet are not accessed again (Kalambe et al., 2019). This proxy is aligned with reuse-based insertion filtering that targets objects "without reuse" (Díaz et al., 2019), and we report both per-slot values and trace averages to summarize long-run efficiency.

D. SCORE-BASED ADMISSION SIGNALS USED BY GUARDRAILS

Guardrails Admission assumes an upstream scoring backbone that assigns a utility score to each admission candidate. For each slot t and each $o \in \mathcal{C}_t$, the backbone produces a scalar score $s_t(o)$, where larger values indicate higher predicted utility for caching (e.g., higher likelihood of near-future reuse or higher predicted popularity). The scoring model is trained and updated online from the request stream using incremental learning, so scoring at slot t is based only on information available up to the end of slot t (Elwell and Polikar, 2011; Xu et al., 2020). The backbone can be any online predictor that emits comparable scores; our contribution treats it as a black box and stabilizes the downstream admission decision. Given candidate scores, we define a slot-specific ranking π_t as a permutation of $\mathcal{C}_t$ that orders candidates in non-increasing score: $s_t(\pi_t(1)) \geq s_t(\pi_t(2)) \geq \cdots$. Section V specifies how the controller converts this ranking into a score threshold and an admission set size, but the preliminaries require only that $s_t(o)$ and π_t satisfy the online-safe constraint stated above and are computed before any insertions derived from slot t are applied.

The first guardrails signal is outcome-based effective precision, which measures how often recent admissions produced subsequent hits. Let F be a feedback window measured in slots; the value of F is specified in Section VI. For the insertions applied at the beginning of slot t, define an indicator $U_t(o) = 1$ if $o \in A_{t-1}^{\mathrm{applied}}$ yields at least one cache hit within the next F slots after insertion, and $U_t(o) = 0$ otherwise. The per-slot effective precision is then $p_t = \frac{\sum_{o\in A_{t-1}^{\mathrm{applied}}} U_t(o)}{\max\{1,|A_{t-1}^{\mathrm{applied}}|\}}$, where the denominator convention avoids division by zero when no insertions were applied. Unlike the global hit ratio, p_t conditions on admitted items and therefore isolates the admission module's realized usefulness. Because reuse may arrive with variable delay, we also define a lag-robust variant. Let $p_t^{(\ell)}$ be computed as above but counting hits observed with an offset of ℓ slots after insertion, for $\ell \in \{0,1,\ldots,K\}$, and set $p_t^{\mathrm{eff}} = \max_{0\leq\ell\leq K} p_t^{(\ell)}$, where K is specified in Section VI. This bounded-delay feedback summarizes admission reliability and is used to regulate future admission pressure.

The second guardrails signal captures score quality, namely how separable the candidate scores are in a given slot. Even with an online learner, a flat or noisy score landscape implies that the induced ranking is unreliable; admitting many candidates in such slots can create insertion bursts and pollution. We therefore summarize score separability through a spread statistic over the candidate score multiset $\{\, s_t(o) \mid o \in \mathcal{C}_t \}$.

Let $Q_\alpha(t)$ denote the α-quantile of these scores at slot t. We define the score spread as $q_t = Q_{0.90}(t) - Q_{0.10}(t)$, which is robust to outliers while reflecting the distance between high and low-scored candidates. Larger q_t indicates clearer separation and hence a more confident ranking, whereas small q_t indicates ambiguity where a strict gate is desirable. For numerical stability, the controller may normalize q_t by a running reference level (e.g., a recent median), but q_t is always computed from online-safe statistics. In Section V, q_t is combined with effective-precision feedback p_t^{eff} to compute admission guardrails that adapt both the admission threshold and the admitted volume.

IV. SYSTEM MODEL AND PROBLEM FORMULATION

This section specifies the system model and the decision interface that Guardrails Admission controls, and it states the admission-stabilization problem studied in the remainder of the paper. The goal is to translate the operational picture in Figure 1 into a minimal formalism: the cache serves an online request stream on the fast path, while an end-of-slot controller computes a gated admission decision that is applied at the next slot boundary.

A. SYSTEM ARCHITECTURE AND EXECUTION TIMELINE

Figure 1 summarizes a two-path architecture that separates request serving from admission control. On the online request serving path, each request in slot t is processed immediately against the current cache state, producing hits and misses and updating the LRU recency structure on hits, as specified in Section III. Misses trigger origin retrieval for service, but they also generate the per-slot candidate set that the controller will later consider for admission. In parallel, the end-of-slot control path aggregates the candidate information and the guardrails signals computed from prior outcomes, then outputs admission parameters for the next slot. Concretely, at the end of slot t the controller determines which candidates from the current slot are eligible and how many will be inserted; these insertions are not performed immediately, but are applied at the beginning of slot $t+1$. Throughout, the cache is treated as a single-node trace-replay model under the same workload trace, and only the admission interface is instrumented for end-of-slot control.

Fixing eviction to LRU is central to the causal interpretation of Figure 1. In ICN systems, admission or placement and eviction or replacement are treated as distinct levers, and holding replacement fixed is a common way to attribute observed hit-ratio changes to admission decisions rather than to a redesigned eviction heuristic (Batool et al., 2024; Rashid et al., 2021). Accordingly, the controller in the end-of-slot path does not modify the LRU recency update performed on hits, nor does it access future requests; it only decides whether and how aggressively to insert candidates that were observed as misses in the past slot. This design keeps the overall system non-anticipative and comparable to the trace-driven evaluation setting where hit probability is defined over a replayed request stream (Panigrahy et al., 2021). Operationally, the separation implies that any improvement in hit ratio or reduction in pollution indicators arises from regulating admission pressure, not from changing how LRU selects victims.

B. ADMISSION-CONTROL INTERFACE AND DECISION VARIABLES

We model admission as a slot-level control action that selects a subset of the current miss candidates. At the end of slot t, the controller outputs an admission budget M_t and a score gate τ_t, and these parameters define the decision set A_t that will be inserted at the beginning of slot $t+1$. Using the upstream scores $s_t(o)$ for candidates $o \in \mathcal{C}_t$ from Section III, the controller first filters candidates through the gate to form an eligible pool, and then admits at most M_t objects by score ranking. Concretely, we define the gated selector in (1). The resulting A_t is the only controllable cache update input. Here, $\text{Top-}M_t(B_t)$ returns the M_t highest-scoring elements in B_t (ties are broken consistently). If $|B_t| < M_t$, all eligible items are selected. By separating eligibility from budget, τ_t controls how permissive admission is, while M_t caps the maximum insertion burst even when many candidates pass the gate.

$$B_t = \{\, o \in \mathcal{C}_t \mid s_t(o) \geq \tau_t \,\}, \quad A_t = \text{Top-}M_t(B_t) \tag{1}$$

The selector in (1) affects cache content only through boundary insertions, while all in-slot hit handling follows fixed LRU. Let $\text{LRU}(\cdot)$ denote the deterministic state-transition operator induced by the LRU discipline when processing the requests of slot t and then applying the decided insertions at the next slot boundary. With one-slot delayed application, the decision A_t computed from slot t is applied before serving slot $t+1$, producing the next-slot state as summarized in (2). Equation (2) abstracts the internal LRU stack operations because eviction is locked and described in Section III. Importantly, the controller cannot influence which resident item is evicted except indirectly, via which objects are inserted and thus become part of the recency stack. Consequently, changes in hit ratio and pollution indicators can be attributed to admission decisions under an unchanged eviction baseline (Batool et al., 2024; Rashid et al., 2021). The delayed application also avoids look-ahead, keeping the overall system non-anticipative over the trace (Panigrahy et al., 2021).

$$\begin{aligned} S_{t+1} &= \text{LRU}(S_t, \mathcal{R}_t, A_t), \\ &\quad \text{with } A_t \text{ applied at the beginning of slot } t+1. \end{aligned} \tag{2}$$

The guardrails controller is intentionally lightweight and treats the scoring model as an upstream component. Any incremental or online learner that outputs per-candidate scores can provide $s_t(o)$, for example, ensemble-based incremental learning that updates under nonstationary streams (Elwell and Polikar, 2011; Xu et al., 2020). The controller itself does not inspect model internals; it consumes only the slot-level reliability feedback p_t^{eff} and score-quality indicator q_t defined in Section III, together with the score list $\{\, s_t(o) \mid o \in \mathcal{C}_t \,\}$ over the current candidates. All inputs are required to be online-safe, meaning they can be computed using data observed up to the end of the current slot and do not rely on future requests

(Panigrahy et al., 2021). As a result, the controller acts as a stabilization layer that regulates how scores are translated into insertions, independently of how scores are learned.

C. STABILIZED SCORE-BASED ADMISSION UNDER FIXED LRU

With the architecture and interface fixed, we formulate the problem addressed by this paper as stabilized score-based admission under fixed LRU eviction. The primary objective is to maximize the hit ratio over the request trace, but the policy must do so in a way that avoids destabilizing insertion behavior. In nonstationary workloads, a scoring model can temporarily produce ambiguous rankings, for example when score separation among candidates is small; admitting purely by rank in such periods can trigger insertion bursts that displace useful content and increase cache pollution. Pollution here refers to wasted cache occupancy by objects that provide little or no subsequent hit benefit, which degrades cache effectiveness under limited capacity (Kalambe et al., 2019). Because eviction remains LRU, these effects arise from the admission side: low-yield insertions increase churn and can cause the cache to lag behind near-future demand. Guardrails therefore treats admission as a control layer that adapts (M_t, τ_t) online to balance hit improvement against insertion and pollution stability under fixed eviction (Rashid et al., 2021).

We express stabilization through explicit constraints and operating limits on the admission controller. First, admission decisions should be reliable in the sense that recently admitted items should deliver hits at a sufficient rate, as captured by the effective-precision feedback p_t^{eff} from Section III. We therefore impose the reliability target in (3), where $p_{\min}$ is a tunable lower bound on admission effectiveness. When p_t^{eff} falls, a stabilizing controller should reduce admission pressure, for instance by shrinking M_t or tightening the gate τ_t. Second, the controller must bound insertion pressure to prevent bursts that can induce churn and wasted occupancy even when scores are temporarily optimistic. In addition, low score quality q_t can motivate raising τ_t even if budget remains available. We enforce the global cap in (4), which limits the maximum number of applied insertions per slot. Both constraints are evaluated online-safe, using only observations available by the end of the current slot, and thus remain compatible with non-anticipative cache operation (Panigrahy et al., 2021).

$$p_t^{\text{eff}} \geq p_{\min} \tag{3}$$

$$|A_t| \leq M_{\max} \tag{4}$$

Beyond hard constraints, we adopt a unified objective that captures the trade-off between hit performance and admission stability over a trace horizon in a trace-driven setting. Let HR denote the hit ratio from Section III, and define the trace averages $\bar{\rho} = \frac{1}{T}\sum_{t=1}^{T} \rho_t$ and $\overline{\text{Poll}} = \frac{1}{T}\sum_{t=1}^{T} \text{Poll}_t$ using the per-slot metrics. The admission-stabilization problem is then expressed in (5) as an optimization over the controller outputs $\{M_t, \tau_t\}$, subject to online-safe operation. Here λ_1 and λ_2 are nonnegative weights that encode the system's tolerance for insertion volume and pollution relative to hit gains. This formulation emphasizes that Guardrails Admission is not a new eviction policy but a stabilizing layer that shapes admission pressure using feedback and score-quality signals. Because (5) is written in terms of auditable counters, Section VI reports insertion and pollution measures in addition to HR and uses component-wise ablation to attribute gains to admission stabilization, consistent with the pollution-as-waste perspective (Kalambe et al., 2019; Rashid et al., 2021).

$$\max_{\{M_t, \tau_t\}} \ \text{HR} - \lambda_1 \bar{\rho} - \lambda_2 \overline{\text{Poll}} \quad \text{s.t. online-safe operation} \tag{5}$$

Algorithm 1 Guardrails Admission with One-Slot Delayed Apply

Require: Slot request sets $\mathcal{R}t$; fixed LRU cache with capacity C; upstream incremental scorer $\mathcal{M}t$; initial controller $(M-1, \tau-1)$; controller parameters $(M_{\max}, \tau_{\min}, \tau_{\max}, \eta_M, \eta_\tau, p_{\text{tar}}, q_{\min})$.

Ensure: Pending admission sets A_t; updated controller states (M_t, τ_t); updated model snapshots $\mathcal{M}_t$.

Initialize $S \leftarrow S_0, A_{-1} \leftarrow \emptyset$.
for $t = 0, 1, 2, \ldots$ **do**
  Apply delayed admission: insert A_{t-1} into S with LRU eviction if needed.
  Initialize $\mathcal{C}_t \leftarrow \emptyset$ and online-safe slot statistics.
  for all $(o_i, \tau_i) \in \mathcal{R}_t$ **do**
    if $o_i \in S_t$ **then**
      serve from cache; update LRU recency state.
    else
      fetch o_i; add o_i to $\mathcal{C}t$.
    end if
    update online-safe statistics/features.
  end for
  Construct candidate features $x_t(o) o \in \mathcal{C}t$ and build slot dataset D_t (online-safe).
  Score candidates using the pre-update snapshot: $s_t(o) \leftarrow \mathcal{M}t - 1(x_t(o))$; compute q_t.
  Compute feedback p_t^{eff} from previously applied admissions (Section III-D).
  Update controller (M_t, τ_t) using the laws in (6) and (7).
  Select A_t using the gated-and-budgeted rule in (1); store A_t as pending.
  Update the scorer after selection: $\mathcal{M}t \leftarrow \text{Update}(\mathcal{M}t - 1, D_t)$.
end for

V. PROPOSED GUARDRAILS ADMISSION MECHANISM

This section presents Guardrails Admission, a stabilization layer placed after scoring and before insertion. Rather than altering the cache dynamics in Section III, the mechanism regulates admission intensity through a slot-level controller that (i) limits how many miss candidates are admitted and

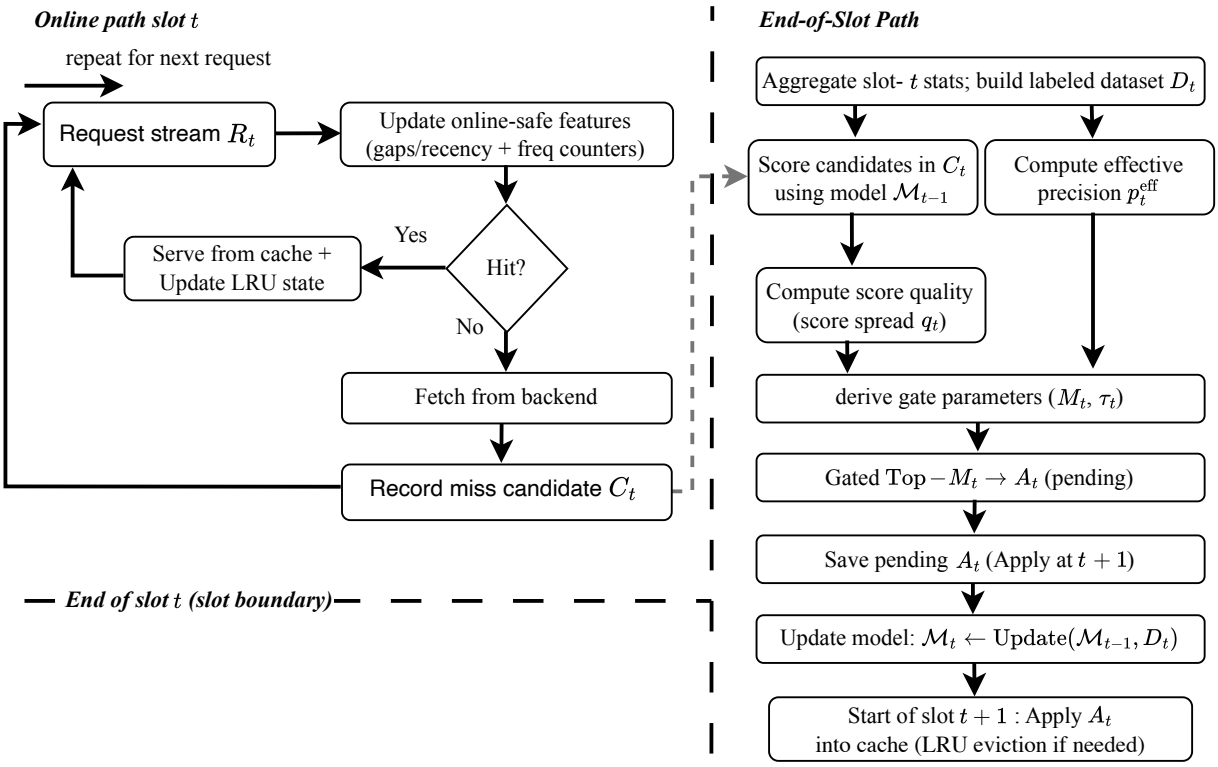

FIGURE 2. **Slot-level execution timeline of the guardrails-aware proactive admission controller with one-slot delayed application.**

(ii) gates admission when scores are unreliable. We describe the causal execution order (Fig. 2 and Algorithm 1), the online-safe candidate representation and incremental scoring backbone, and the feedback laws that adapt the budget and gate.

A. SLOT-LEVEL DATAFLOW AND EXECUTION ORDER

Guardrails Admission operates on two synchronized paths. During slot t, the online serving path processes the request set $\mathcal{R}_t$ using the cache state obtained after applying the pending admissions from slot $t-1$. Hits are served locally and update the recency state, whereas misses trigger backend fetches and register objects into the slot's candidate container $\mathcal{C}_t$ defined in Section III-A. No admission is executed mid-slot; instead, the end-of-slot control path runs after all requests in $\mathcal{R}_t$ are served. At the slot boundary, the controller (i) scores the miss candidates in $\mathcal{C}_t$, (ii) evaluates the feedback signals, and (iii) selects the pending admission set A_t, which is inserted only at the beginning of slot $t+1$. Figure 2 depicts this timeline, while Algorithm 1 provides the procedural realization by sequencing apply, serve-and-log, score-and-control, and defer. This separation makes the intervention auditable: aside from the standard recency update on hits, all cache-state changes originate from delayed admissions.

A key discipline is predict-then-update. Candidate scoring for slot t is performed with the model snapshot available before the slot ends, denoted $\mathcal{M}_{t-1}$, and only after A_t has been committed do we update the scorer to $\mathcal{M}_t$ using labeled instances accumulated in the same slot. This prevents intra-slot label leakage: training on outcomes observed in slot t cannot influence the ranking of candidates generated from the misses of slot t. The discipline is consistent with non-anticipative cache control, where decisions are made without future request knowledge and can therefore be evaluated via trace replay (Panigrahy et al., 2021). Moreover, keeping the eviction rule unchanged enables attribution: differences in hit ratio, insertion intensity, and pollution indicators arise from admission regulation rather than replacement changes, matching the modular separation of admission and eviction emphasized in caching architectures (Batool et al., 2024; Rashid et al., 2021).

B. ONLINE-SAFE CANDIDATE REPRESENTATION AND INCREMENTAL SCORING BACKBONE

To avoid notation collisions with the admission budget M_t, we denote the upstream scoring backbone at slot granularity by $\mathcal{M}_t$ and represent each miss candidate $o \in \mathcal{C}_t$ with an online-safe feature vector $x_t(o)$. Feature extraction is restricted to observations available up to the end of slot t and is assembled from lightweight recency and popularity statistics. For recency, we follow the gap-feature construction of Xu et al. (2020) and use $L = 6$ gap components, denoted $\text{Gap}_1, \ldots, \text{Gap}_L$, where Gap_j is the time difference between the current request timestamp (the most recent request of o

observed up to the end of slot t) and the j-th most recent previous request timestamp of the same object (Xu et al., 2020). For popularity, we use windowed request-frequency counters computed at slot granularity: f_{short} counts requests to o over the most recent 1 slot, f_{mid} over the most recent 7 slots, f_{long} over the most recent 30 slots, and f_{cum} is the cumulative request count over the replay prefix. The use of frequency features over recent windows follows the windowed-frequency signal used for cache learning and selection in Zhou et al. (2025). Together, the main IL variants use a 10-dimensional feature vector (6 gaps and 4 frequency counters) for each $o \in \mathcal{C}_t$.

After the slot completes, each candidate is scored as $s_t(o) = \mathcal{M}_{t-1}(x_t(o))$, producing the ranking signal consumed by the gated Top-M_t selector in (1). In addition, the slot boundary is where we assemble the training dataset D_t used to update the backbone, ensuring that model updates do not interfere with same-slot scoring. This interface keeps the guardrails mechanism agnostic to the specific learner, while guaranteeing that every score and feature used for selecting A_t is online-safe and reproducible under trace replay in our evaluation pipeline.

The incremental learner updates the scoring backbone from streaming evidence without breaking the predict-then-update discipline. After slot t completes, we assemble a slot training dataset D_t from online-safe logs as feature-label pairs for the current workload, where labels are derived from the same reuse outcomes used for feedback in Section III-D. The backbone is then updated as $\mathcal{M}_t \leftarrow \text{Update}(\mathcal{M}_{t-1}, D_t)$, and the new snapshot is used only starting in slot $t+1$. This workflow matches incremental ensemble learning principles such as Learn++.NSE, which are designed for nonstationary streams by continually adding or reweighting weak learners as new data arrive (Elwell and Polikar, 2011). The proposed guardrails remain compatible with other online incremental learners used in edge caching systems, as long as they expose the same score interface $s_t(o)$ and support per-slot updates under the online-safe constraint in trace replay (Xu et al., 2020).

C. GUARDRAILS CONTROLLER: BUDGET-AND-GATE ADAPTATION

Guardrails implements admission as a control layer that adapts two parameters, (M_t, τ_t), from outcome-based reliability and score separability. This design follows a common pattern in admission work: the system regulates insertion intensity through gates or probabilities that respond to workload dynamics (Qi et al., 2019). For example, ED-FGPC adjusts a popularity threshold using incoming interests and cache constraints, demonstrating that feedback-driven gating can improve hit ratio under time-varying demand (Banerjee & Biswash, 2024). Filter similarly uses a time-windowed frequency statistic as an online admission gate in Named Data Networking (NDN) routers (Takemasa et al., 2019). In CDN and datacenter settings, SLAP and CacheSack couple learned scoring with operating-condition or benefit feedback to stabilize insertions and reduce write-related costs (Liu et al., 2024; Yang et al., 2023). Motivated by these patterns, we couple the effective precision feedback p_t^{eff} and the score-quality signal q_t to adapt the admission budget and the score gate, thereby suppressing bursts when rankings are ambiguous while remaining responsive when decisions are reliable.

Formally, at the end of slot t we update the controller state (M_t, τ_t) from the previous state and the two feedback signals. We adopt a saturated additive form to avoid abrupt swings: $\text{clip}_{[a,b]}(x) = \min\{\max\{x,a\},b\}$ enforces hard bounds, $[z]_+ = \max\{z,0\}$ isolates deficits, and $\text{Round}(\cdot)$ maps the budget to an integer. The budget update reacts primarily to outcome reliability by expanding when p_t^{eff} exceeds a target p_{tar} and contracting otherwise. The score gate reacts to both reliability and separability by tightening when either p_t^{eff} drops or q_t falls below $q_{\min}$, which directly shrinks the eligible pool used by the selector in (1). The resulting update laws are summarized in (6) and (7) and are parameterized only by step sizes (η_M, η_τ) and bounds $(M_{\max}, \tau_{\min}, \tau_{\max})$, enabling robust and stable tuning across traces and operating conditions in practice.

$$M_t = \text{Round}\Big(\text{clip}_{[0,M_{\max}]}\big(M_{t-1} + \eta_M\,(p_t^{\text{eff}} - p_{\text{tar}})\big)\Big) \quad (6)$$

$$\tau_t = \text{clip}_{[\tau_{\min},\tau_{\max}]}\Big(\tau_{t-1} + \eta_\tau\,(p_{\text{tar}} - p_t^{\text{eff}}) + \eta_\tau\,[\,q_{\min} - q_t\,]_+\Big) \quad (7)$$

Two stabilizing mechanisms arise from the coupling in (6) and (7). First, when the effective precision signal degrades, both the budget and the gate move in the conservative direction, which reduces the number of admitted objects and increases the expected hit yield per insertion. This mirrors the empirical rationale in learned admission studies that jointly report higher hit rate with lower write traffic, interpreted as fewer low-yield insertions (Liu et al., 2024). Second, when score separability is low, the gate term based on q_t increases τ_t even if p_t^{eff} has not yet dropped, providing an anticipatory guard against insertion bursts driven by ambiguous rankings. In trace-driven settings, uncontrolled low-yield insertions are closely related to cache pollution, understood as wasted occupancy by objects that are no longer accessed (Kalambe et al., 2019), and to churn phenomena where wasteful turnover degrades hit probability (Yoji et al., 2020). Finally, the loop is closed through (1): the updated (M_t, τ_t) determines A_t which is applied one slot later, thereby stabilizing admissions under the cache dynamics defined in Section III.

VI. EXPERIMENTAL SETUP AND EVALUATION

This section describes the trace-driven evaluation protocol and the experimental settings used to assess Guardrails Admission. The goal is to make the results reproducible and comparable across methods by fixing the workload prefix, slotting and warm-up, cache-size sweep, controller parameters, and baseline hyperparameters. All performance metrics and the cache/admission semantics follow the definitions introduced in Section III.

A. TRACES AND EXPERIMENT SETUP

1) Traces and trace-driven replay

We evaluate on two real request traces that are widely used in caching studies. The first workload is the Wikipedia Sep-2007 trace released by Urdaneta *et al.* (Urdaneta et al., 2009). From this trace, we replay a prefix of 9M requests, which contains 1,527,731 distinct objects (internal count on the replay prefix). The second workload is the Wikipedia CDN trace distributed with the NSDI'20 LRB study (Song et al., 2020). We replay a prefix of 10M requests with 2,015,444 distinct objects (internal count on the replay prefix). For both traces, requests are replayed in timestamp order against a single-cache simulator, and decisions are computed without any look-ahead, consistent with non-anticipative trace-driven evaluation (Panigrahy et al., 2021).

2) Slotting, warm-up, and one-slot delayed apply

Time is partitioned into slots of 100k requests per slot. To reduce transient effects from the empty initial cache state, we apply a warm-up of 1M requests (10 slots), and report all metrics on the remaining suffix of the replay. Under this configuration, the evaluation horizon contains 80 reporting slots for Wikipedia Sep-2007 and 90 reporting slots for Wiki2018. Admission decisions are computed and applied using the one-slot delayed rule defined in Section III-B: the pending admission set A_t is computed at the end of slot t from the miss-candidate set $\mathcal{C}_t$ and is inserted at the beginning of slot $t+1$.

3) Cache model, capacity sweep, and fixed eviction for fairness

We use an object cache where capacity C is measured in number of objects, and all policies operate at object granularity. Following the capacity-regime practice used in incremental-learning caching evaluation (Xu et al., 2020), we sweep cache size from 0.8% to 5% of the distinct objects in the replay prefix. Concretely, we evaluate $C \in \{0.8\%, 1\%, 2\%, 3\%, 4\%, 5\%\}$ of the prefix catalog size, which produces the HR–capacity curves shown in Fig. 4a and Fig. 5a. To ensure fair admission comparisons, all admission variants share the same cache dynamics from Section III-B, and the eviction policy is unchanged across methods; the only varying component is the admission decision.

4) Horizon parameters for admission-side outcome metrics

Admission-side outcome metrics in Fig. 4 and Fig. 5 are computed with fixed horizons that are shared by all policies. The hit-yield Y_t and pollution proxy Poll_t use a post-admission horizon of $W = 1$ slot. Effective precision feedback uses a window of $F = 2$ slots, and lag robustness is computed with $K = 2$ as defined in Section III-D. These values are held constant across both traces and across all guardrails ablations.

5) Guardrails parameters and implementation settings

Guardrails Admission is instantiated exactly as described in Algorithm 1 and in the update laws (6) and (7). We initialize the admission budget at $M_{-1} = \lceil 0.08C \rceil$ and bound its operating range by $M_{\max} = \lceil 0.2C \rceil$. The target effective-precision level is set to $p_{\mathrm{tar}} = 0.12$. For the score-quality signal, we use the spread statistic defined in Section III-D, and apply lightweight smoothing (EMA factor 0.2) when updating the gate to avoid abrupt slot-to-slot swings. Additional gating constants used by the implementation include a minimum gate floor of 0.005, a maximum gate ceiling of 0.2, and a top-percent screening ratio of 0.05, which are kept fixed across traces. Unless stated otherwise, the same controller settings are used for all cache sizes in the sweep.

For the incremental-learning backbone used by IL_no_guard and IL_Guard, both variants share the same online-safe candidate representation and predict-then-update discipline described in Section V. In particular, the scorer is updated once per slot using streaming incremental learning principles (Elwell and Polikar, 2011; Xu et al., 2020), and the main IL runs use the 10-feature representation introduced in Section V-B, combining six gap features with $L = 6$ (Xu et al., 2020) and four windowed frequency counters (Zhou et al., 2025).

6) Baselines and hyperparameters

We compare against representative recency- and frequency-oriented baselines, as well as a learning-based baseline, using standard parameterizations. LRU and LRU-K follow the definitions in (O'Neil, 1993), with LRU-K evaluated at $k = 2$. TinyLFU is implemented using a Count-Min Sketch with 16384×4 counters and a doorkeeper of 65536 bits, with periodic reset at $\max(100k, 10C)$ requests, following standard TinyLFU configurations (Einziger et al., 2017). The GDBT baseline uses 30 estimators and is updated every 1M requests (Berger et al., 2018). All methods are evaluated under identical trace prefixes, slotting, warm-up, and cache-size sweep.

B. EVALUATION PROTOCOL AND RESULTS DISCUSSION

1) Reporting methodology

We report hit ratio (HR) as the primary effectiveness metric and present HR–capacity curves to assess robustness across the cache-size regime. To explain *why* HR changes under different admission behaviors, we additionally report three admission-side diagnostics defined in Section III-C and Section III-D: (i) the insertion rate (normalized admission intensity), (ii) the pollution proxy Poll_t (never-reused admissions within horizon W), and (iii) hit-yield Y_t (hits generated per admitted object within horizon W). For each trace, Fig. 4 and Fig. 5 provide a 2×2 panel summary: panel (a) plots HR versus cache size; panel (b) plots insertion rate; panel (c) plots the pollution proxy; and panel (d) plots hit-yield. All curves are computed after the warm-up prefix, using the same horizons and normalization across methods.

2) Wikipedia Sep-2007 results

Fig. 4a compares HR across cache sizes for LRU, LRU-K, TinyLFU, GDBT, IL_no_guard, and IL_Guard. Across the entire capacity sweep, IL_Guard achieves the highest HR, with IL_no_guard typically forming the next-best curve.

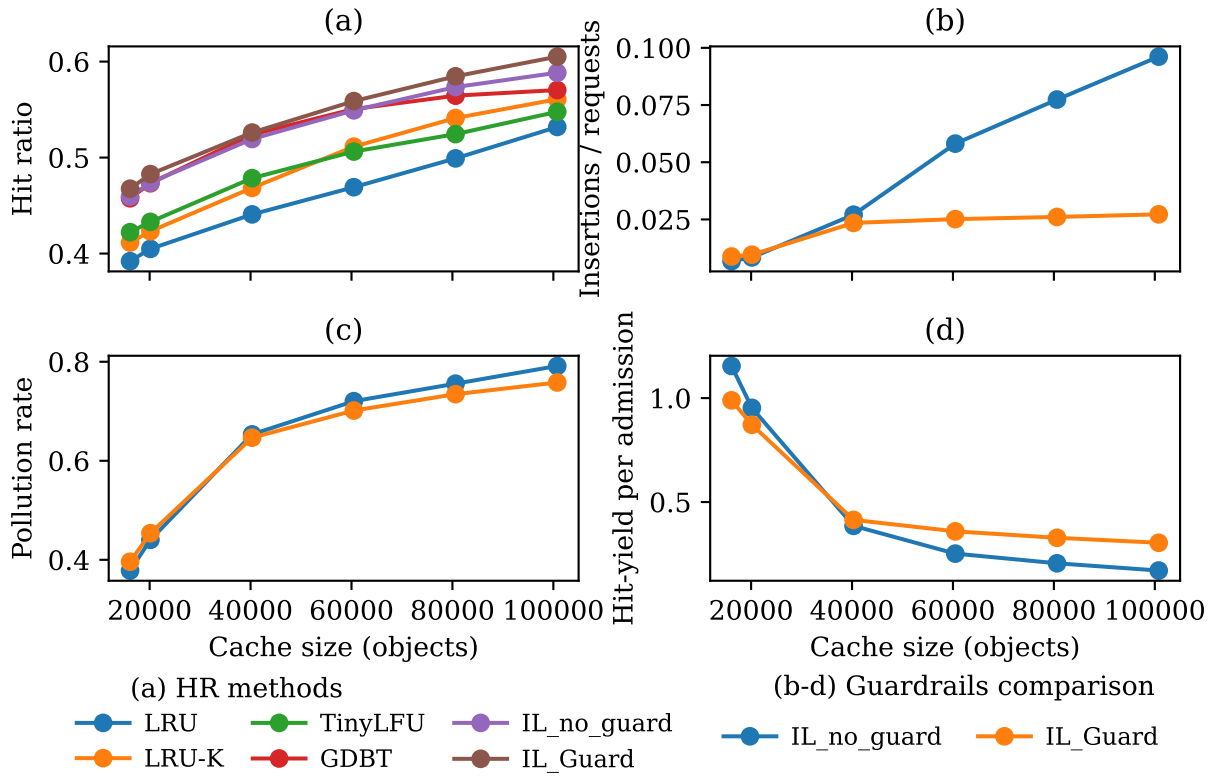

FIGURE 3. System architecture and cache-control modules at an edge node

The heuristic baselines (LRU, LRU-K, and TinyLFU) occupy lower HR ranges, while GDBT improves over pure heuristics but does not match the IL variants on this trace. Importantly, the HR improvement of IL_Guard over IL_no_guard is consistent across cache sizes, suggesting that the guardrails layer contributes beyond the scoring backbone itself rather than benefiting only at a single operating point.

The admission-side panels clarify the mechanism. Fig. 4b shows that IL_no_guard increases insertion intensity as cache size grows, indicating that rank-based admission without stabilization tends to admit a larger fraction of the miss-candidate pool at higher capacities. In contrast, IL_Guard keeps insertion rate comparatively stable, reflecting the effect of the budget-and-gate control in suppressing admission bursts. This stabilization aligns with the pollution and yield behavior: Fig. 4c shows that the pollution proxy increases with cache size for both variants, but IL_Guard maintains a lower pollution level than IL_no_guard at moderate and large capacities. Correspondingly, Fig. 4d shows higher hit-yield under IL_Guard, especially when cache size increases. Taken together, the panel set indicates that the HR gain in Fig. 4a is accompanied by a shift toward fewer low-yield admissions, which is consistent with the interpretation of pollution as wasted occupancy by objects that are not accessed again (Kalambe et al., 2019).

3) Wiki2018 results

Fig. 5a reports the HR–capacity curves on the Wiki2018 trace. The relative ordering is consistent with the Sep-2007 results: the two IL variants lead the comparison, and IL_Guard remains the best-performing configuration across the cache-size sweep. Compared to IL_no_guard, the margin of IL_Guard becomes more visible at larger cache sizes, where uncontrolled insertion pressure can accumulate and reduce the average utility of admitted objects.

The guardrails effect is again explained by insertion stability and admission efficiency. Fig. 5b shows that IL_no_guard exhibits a pronounced increase in insertion rate as cache size grows, whereas IL_Guard limits this growth. Fig. 5c indicates that IL_Guard reduces the fraction of never-reused admissions relative to IL_no_guard, and Fig. 5d shows that IL_Guard achieves consistently higher hit-yield per admission, particularly at moderate and large capacities. The combined evidence across Fig. 5a–Fig. 5d supports the same qualitative mechanism as in Fig. 4: stabilizing admission pressure improves the yield of admitted objects and reduces pollution, which in turn is reflected in the HR curve.

4) Guardrails ablation across components

To isolate the effect of the two guardrails signals, we conduct an ablation study in which only one of the two control components is enabled. Fig. 6 (Wiki2018) and Fig. 7 (Wikipedia Sep-2007) compare four configurations: No-

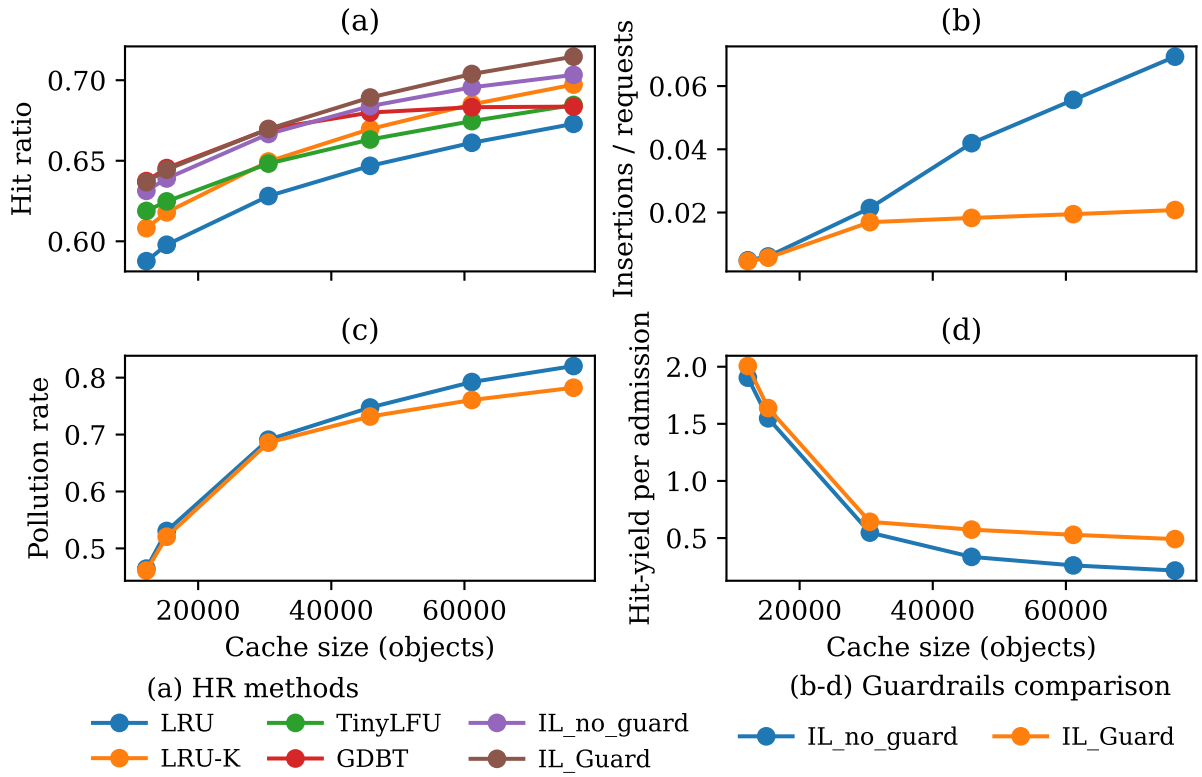

FIGURE 4. System architecture and cache-control modules at an edge node

Guard, Precision-only, Quality-only, and Full-Guard. On both traces, Full-Guard yields the highest HR across the cache-size sweep, indicating that combining outcome-based reliability feedback with score-quality gating provides the most consistent benefit. Precision-only and Quality-only each improve over No-Guard, but the gap to Full-Guard remains visible at moderate and large cache sizes. This pattern suggests that the two signals address complementary failure modes: outcome feedback reacts to low-yield admissions after they occur, while score-quality gating reduces exposure to ambiguous score landscapes where rank-only admission may be unstable.

C. COMPUTATIONAL OVERHEAD PROFILING

Because Guardrails Admission introduces end-of-slot computation in addition to candidate scoring, we report overhead measurements focused on the online-control path that runs at slot boundaries. We measure the end-of-slot time as the wall-clock duration of feature construction, model update, candidate scoring, and admission control (gating and Top-M_t selection) executed once per slot. To keep overhead reporting compact and interpretable, we profile overhead at the smallest cache size in the sweep (0.8% of distinct objects), where candidate sets are typically largest relative to cache capacity and where admission control is most constrained.

Fig. 8 plots the empirical CDF of end-of-slot time for GDBT, IL_no_guard, and IL_Guard on both traces. The distributional view highlights both typical cost and tail behavior: the IL variants complete end-of-slot processing substantially faster than GDBT, while IL_Guard remains close to IL_no_guard, indicating that the guardrails layer adds limited additional overhead on top of incremental scoring.

TABLE 1. End-of-slot control overhead on Wikipedia Sep-2007 (cache size set to 0.8% of the prefix catalog, C=12,221 objects; 80 reporting slots after warm-up).

Policy	Avg (s/slot)	P95 (s/slot)	P99 (s/slot)	Score (μs/cand)
GDBT	25.96	60.66	77.75	443.7
IL-no-guard	4.15	11.43	26.99	17.2
IL-Guard	4.48	12.46	37.70	17.2

TABLE 2. End-of-slot control overhead on Wiki2018 (cache size set to 0.8% of the prefix catalog, C=16,123 objects; 90 reporting slots after warm-up).

Policy	Avg (s/slot)	P95 (s/slot)	P99 (s/slot)	Score (μs/cand)
GDBT	26.06	71.89	109.78	306.8
IL-no-guard	5.40	13.75	26.01	17.6
IL-Guard	5.19	14.66	17.98	17.6

Tables 1 and 2 summarize the same overhead using average and tail percentiles (P95 and P99) of end-of-slot time, together with the average scoring cost per candidate normalized by $|\mathcal{C}_t|$. On both traces, IL_Guard and IL_no_guard have similar per-candidate scoring time, reflecting their shared backbone, while GDBT incurs substantially higher per-candidate scoring cost. Detailed breakdowns of per-stage timings and memory footprint are reported in the supplementary material.

ACKNOWLEDGMENT

The preferred spelling of the word "acknowledgment" in American English is without an "e" after the "g." Use the singular heading even if you have many acknowledgments. Avoid expressions such as "One of us (S.B.A.) would like to thank . . . ." Instead, write "F. A. Author thanks . . . ." In most cases, sponsor and financial support acknowledgments are placed in the unnumbered footnote on the first page, not here.

REFERENCES

FIRST A. AUTHOR received the B.S. and M.S. degrees in aerospace engineering from the University of Virginia, Charlottesville, in 2001 and the Ph.D. degree in mechanical engineering from Drexel University, Philadelphia, PA, in 2008.

From 2001 to 2004, he was a Research Assistant with the Princeton Plasma Physics Laboratory. Since 2009, he has been an Assistant Professor with the Mechanical Engineering Department, Texas A&M University, College Station. He is the author of three books, more than 150 articles, and more than 70 inventions. His research interests include high-pressure and high-density nonthermal plasma discharge processes and applications, microscale plasma discharges, discharges in liquids, spectroscopic diagnostics, plasma propulsion, and innovation plasma applications. He is an Associate Editor of the journal *Earth, Moon, Planets*, and holds two patents.

Dr. Author was a recipient of the International Association of Geomagnetism and Aeronomy Young Scientist Award for Excellence in 2008, and the IEEE Electromagnetic Compatibility Society Best Symposium Paper Award in 2011.

SECOND B. AUTHOR (M'76–SM'81–F'87) and all authors may include biographies. Biographies are often not included in conference-related papers. This author became a Member (M) of IEEE in 1976, a Senior Member (SM) in 1981, and a Fellow (F) in 1987. The first paragraph may contain a place and/or date of birth (list place, then date). Next, the author's educational background is listed. The degrees should be listed with type of degree in what field, which institution, city, state, and country, and year the degree was earned. The author's major field of study should be lower-cased.

The second paragraph uses the pronoun of the person (he or she) and not the author's last name. It lists military and work experience, including summer and fellowship jobs. Job titles are capitalized. The current job must have a location; previous positions may be listed without one. Information concerning previous publications may be included. Try not to list more than three books or published articles. The format for listing publishers of a book within the biography is: title of book (publisher name, year) similar to a reference. Current and previous research interests end the paragraph.

The third paragraph begins with the author's title and last name (e.g., Dr. Smith, Prof. Jones, Mr. Kajor, Ms. Hunter). List any memberships in professional societies other than the IEEE. Finally, list any awards and work for IEEE committees and publications. If a photograph is provided, it should be of good quality, and professional-looking. Following are two examples of an author's biography.

THIRD C. AUTHOR, JR. (M'87) received the B.S. degree in mechanical engineering from National Chung Cheng University, Chiayi, Taiwan, in 2004 and the M.S. degree in mechanical engineering from National Tsing Hua University, Hsinchu, Taiwan, in 2006. He is currently pursuing the Ph.D. degree in mechanical engineering at Texas A&M University, College Station, TX, USA.

From 2008 to 2009, he was a Research Assistant with the Institute of Physics, Academia Sinica, Tapei, Taiwan. His research interest includes the development of surface processing and biological/medical treatment techniques using nonthermal atmospheric pressure plasmas, fundamental study of plasma sources, and fabrication of micro- or nanostructured surfaces.

Mr. Author's awards and honors include the Frew Fellowship (Australian Academy of Science), the I. I. Rabi Prize (APS), the European Frequency and Time Forum Award, the Carl Zeiss Research Award, the William F. Meggers Award and the Adolph Lomb Medal (OSA).